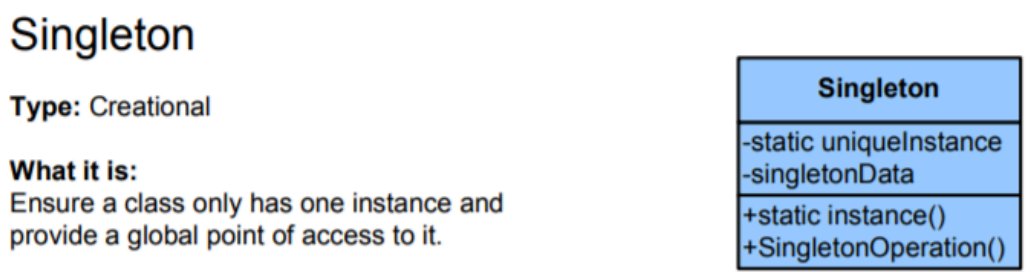
Design Patterns

Note this is not a complete list of the design patterns

Creational

Singleton

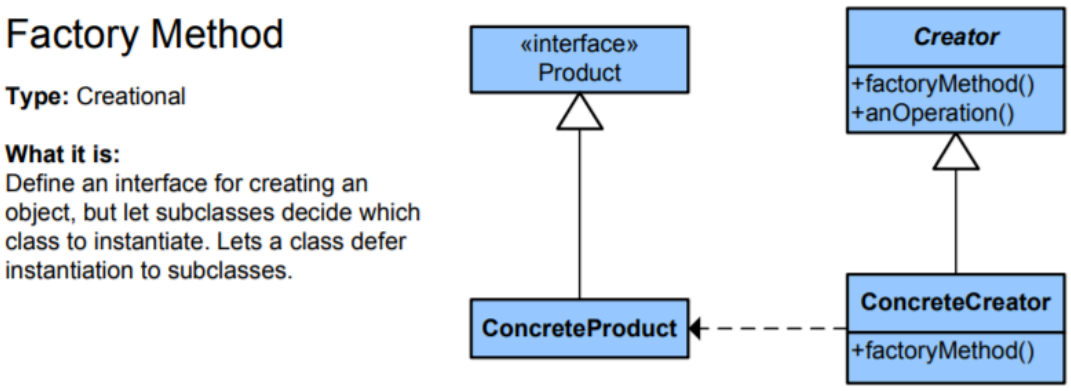
Components: Singleton



Factory Method

Components

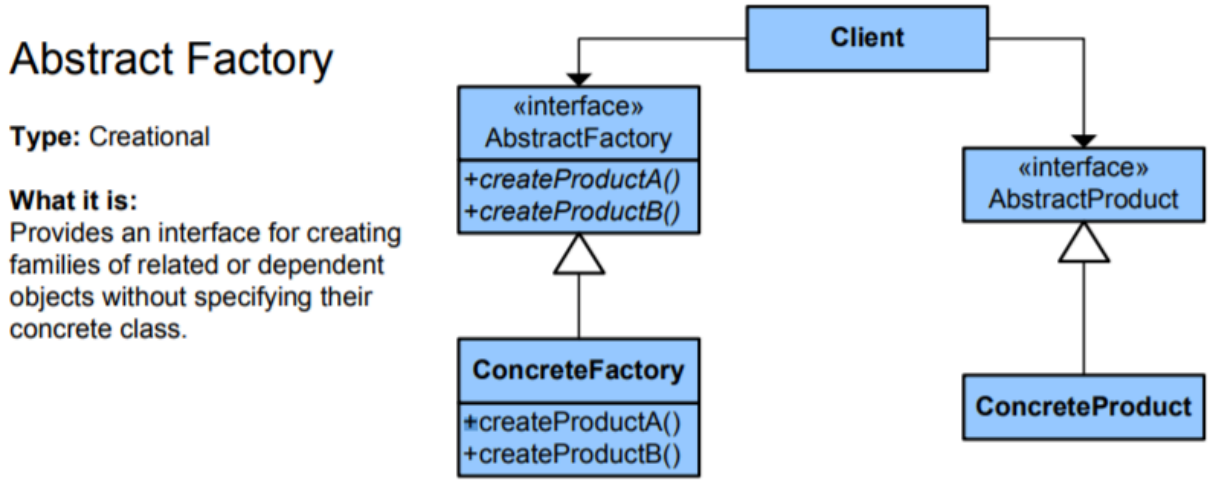
- Product
- Factory



Abstract Factory

Components:

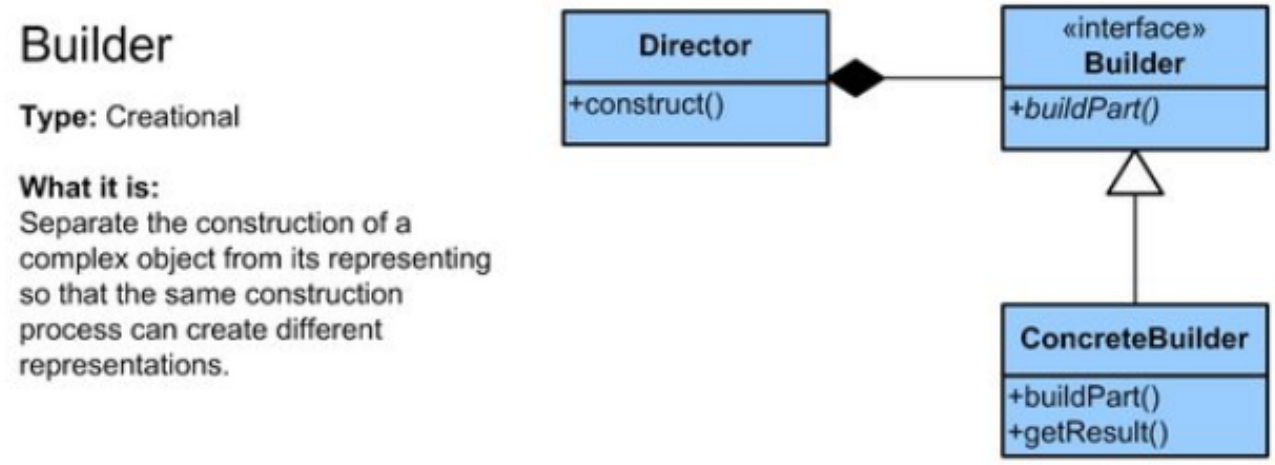
- Abstract Product: Abstraction for products
- Concrete Products : Different families & types of products to be made
- Abstract Factory: Abstraction for factories
- Concrete Factories: Creates different types of products of a particular family



Builder

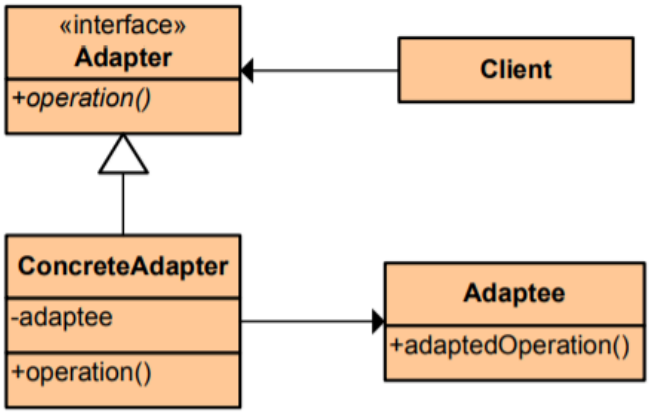
Components:

- Director:
- Builder:
- buildPart()



Structural

Adapter

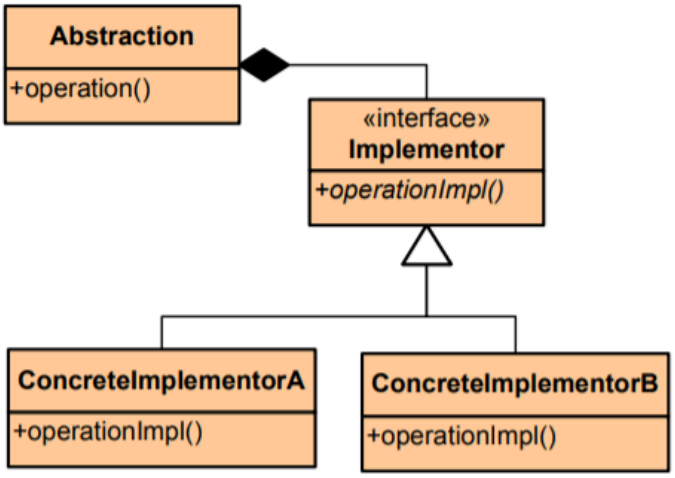


Adapter

Type: Structural

What it is:
Convert the interface of a class into another interface clients expect. Lets classes work together that couldn't otherwise because of incompatible interfaces.

Bridge

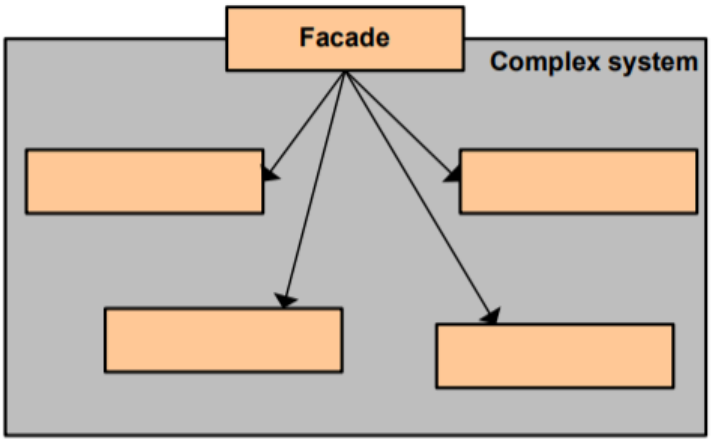


Bridge

Type: Structural

What it is:
Decouple an abstraction from its implementation so that the two can vary independently.

Facade



Facade

Type: Structural

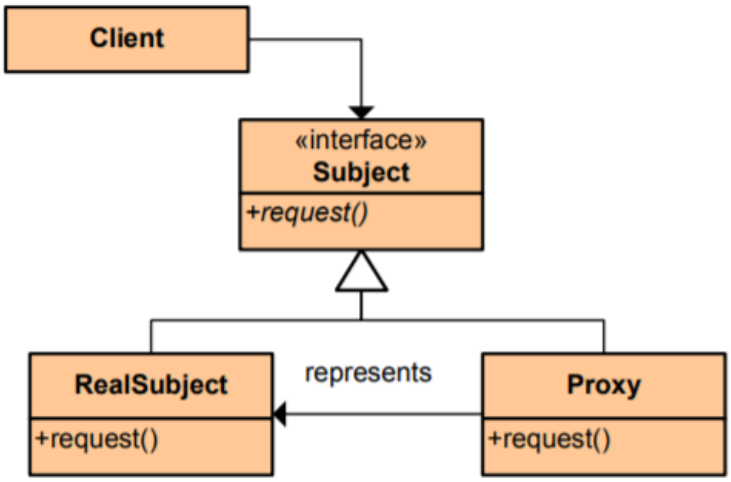
What it is:
Provide a unified interface to a set of interfaces in a subsystem. Defines a high-level interface that makes the subsystem easier to use.

Proxy

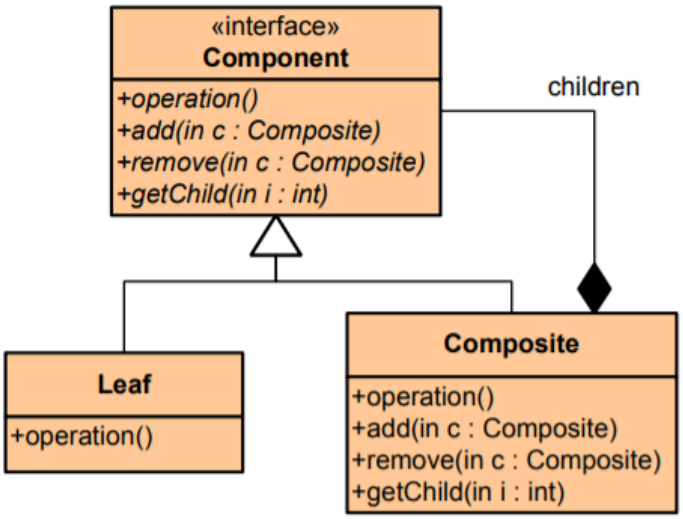
Proxy

Type: Structural

What it is:
Provide a surrogate or placeholder for another object to control access to it.



Composite



Composite

Type: Structural

What it is:
Compose objects into tree structures to represent part-whole hierarchies. Lets clients treat individual objects and compositions of objects uniformly.

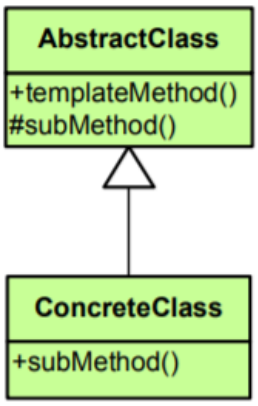
Behavioural

Template Method

Template Method

Type: Behavioral

What it is:
Define the skeleton of an algorithm in an operation, deferring some steps to subclasses. Lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.

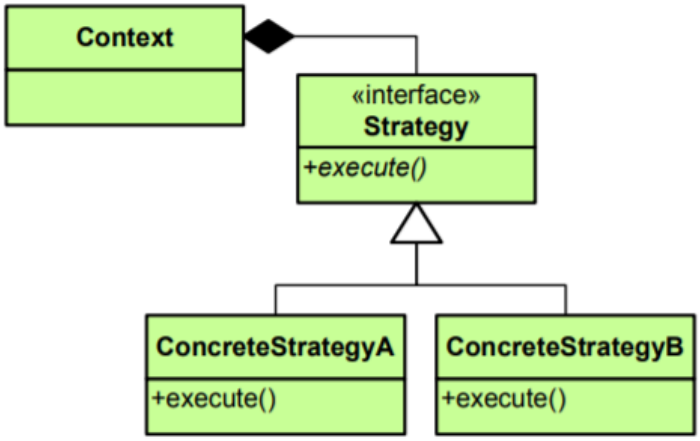


Strategy

Strategy

Type: Behavioral

What it is:
Define a family of algorithms, encapsulate each one, and make them interchangeable. Lets the algorithm vary independently from clients that use it.

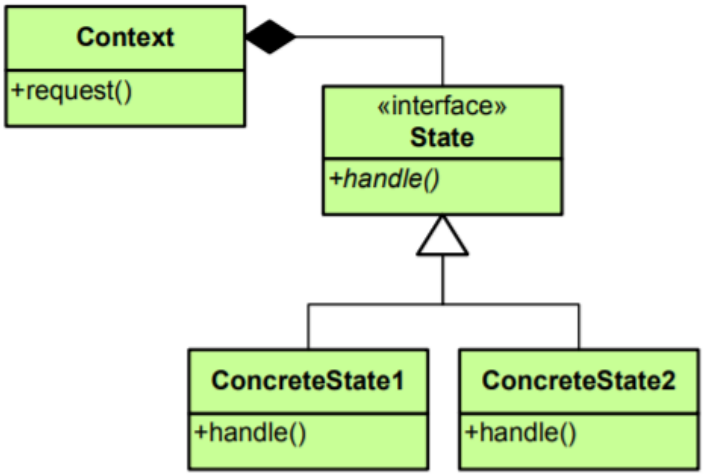


State

State

Type: Behavioral

What it is:
Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.

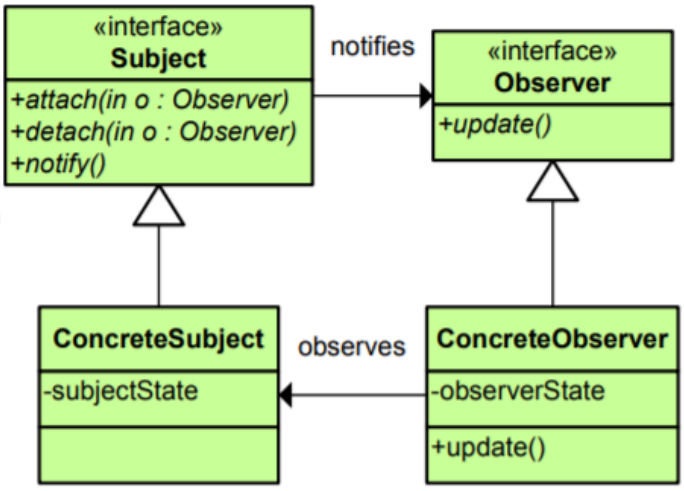


Observer

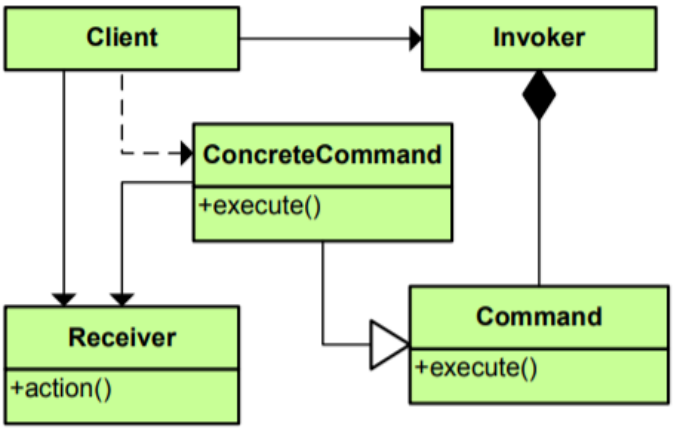
Observer

Type: Behavioral

What it is:
Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.



Command



Command

Type: Behavioral

What it is:
Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.